HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

GRADUATION AND RESEARCH 2

Odoo-based Retail Fashion Management Solution

Instructor: M.S. Lê Tấn Hùng

Student: Bùi Hà My - 20225987

Ha Noi, December 2025

Contents

Executive Summary **2**

1 Business Architecture **3**
1.1 Business Context & Vision 3
1.2 Capability Map diagram 4
1.3 AS-IS Business Problems 5
1.4 Scope & Assumptions 6
1.5 TO-BE Business Process 7

2 Application Architecture **9**
2.1 Application Landscape 9
2.2 Customization Strategy & Application Extensions 11

3 Data Architecture **14**
3.1 Core Data Entities 14
3.2 Data Relationships Traceability 16
3.3 Data Governance & Data Quality Principles 17

4 Technology Architecture **19**
4.1 Technology Architecture Overview 19
4.2 Deployment and Configuration 21

5 Architecture Summary and Conclusion **24**
5.1 Architecture Summary 24
5.2 Key Architectural Decisions 24
5.3 Alignment with Business Objectives 24
5.4 Limitations and Future Evolution 25
5.5 Conclusion 25

Appendix **26**

EXECUTIVE SUMMARY

This report presents the Enterprise Architecture (EA) design of a Lean ERP solution for a retail fashion store implemented on Odoo 18. The target business is a physical retail store operating with direct, over-the-counter sales, a single warehouse, and products with multiple variants such as size and color.

In the current operational state, sales staff face difficulties in identifying real-time stock availability at the variant level, leading to overselling, inaccurate inventory records, and inefficient sales transactions. In addition, the default ERP user interface exposes many unnecessary fields, increasing complexity and the risk of user errors during sales operations.

To address these issues, a Lean ERP approach is adopted, focusing on stabilizing the core retail sales process. The proposed solution improves variant-level inventory visibility, prevents overselling at the point of sale, and simplifies the sales user interface. The implementation follows the Enterprise Architecture 4-layer model, covering Business, Application, Data, and Technology perspectives.

The system is built on standard Odoo modules, complemented by a custom extension module that introduces stock validation logic and user interface simplification without modifying core functionality. Inventory accuracy is ensured through validation at both the user interaction and sales order confirmation stages, while maintaining a single source of truth for transactional data.

Overall, this project demonstrates a practical application of Enterprise Architecture principles in a retail context by delivering a focused, maintainable solution that addresses immediate business needs and establishes a foundation for future enhancements.

1 Business Architecture

1.1 Business Context & Vision

1.1.1 Business Context

The target enterprise is a retail fashion store operating a physical sales counter with direct, walk-in customers. The store primarily sells fashion items that are highly variant-driven, with each product typically available in multiple sizes and colors. As a result, inventory complexity is concentrated not at the product level, but at the variant level, where each size–color combination must be managed independently.

Operationally, the store maintains a single warehouse that serves as both the storage and fulfillment point for retail sales. Sales staff interact directly with customers at the counter and are responsible for creating sales orders during the purchasing process. Due to the small-scale nature of the business, sales staff often perform multiple roles, including customer interaction and order processing, while warehouse activities such as receiving goods and inventory adjustments are handled separately.

In this context, the point of sale becomes a critical control point for inventory accuracy. Any error in determining variant availability at the moment of sale can immediately result in overselling, customer dissatisfaction, and discrepancies between physical stock and system records.

1.1.2 Business Vision

The business vision guiding this project is to establish a lean, reliable, and user-friendly retail sales operation that ensures accuracy without increasing operational complexity. Rather than adopting a fully automated or feature-heavy ERP approach, the enterprise prioritizes stabilizing its core sales process before expanding into more advanced automation.

The key elements of this vision include:

- **Accuracy at the point of sale**
  Sales staff must be able to see correct, real-time stock availability for each product variant and be prevented from selling items that are unavailable.

- **Operational simplicity**
  The sales interface should present only the information and actions necessary for retail transactions, reducing cognitive load and minimizing user errors.

- **Process discipline through system enforcement**
  Business rules such as “do not sell beyond available stock” should be enforced by the system rather than relying on staff experience or manual checks.

- **Scalability through phased implementation**
  The current implementation focuses on core stability—variant-level stock visibility and overselling prevention—while leaving room for future enhancements such as automated delivery confirmation, invoicing, and payment integration.

1.1.3 Business Objectives

The business vision is translated into the following concrete objectives:

Objective	Description
Prevent overselling	Ensure that no sales order can be confirmed if variant-level stock is insufficient
Improve inventory accuracy	Maintain consistency between physical stock and system records
Speed up retail transactions	Reduce time spent checking stock and correcting errors during sales
Reduce user errors	Simplify the sales interface to support frontline staff
Build a stable ERP foundation	Prepare the system for future automation phases

Table 1: Business Objectives and Descriptions

1.2 Capability Map diagram

1.2.1 Business Capability Map

Capability Identification Principles

The capability map is designed according to the following principles:

- Capabilities are defined at a stable and high level, independent of user interface or technical implementation.
- Only capabilities within the project scope are included.
- Capabilities reflect real operational needs of a single-store retail fashion business.
- Advanced or out-of-scope capabilities (e.g., advanced accounting, omnichannel sales) are intentionally excluded.

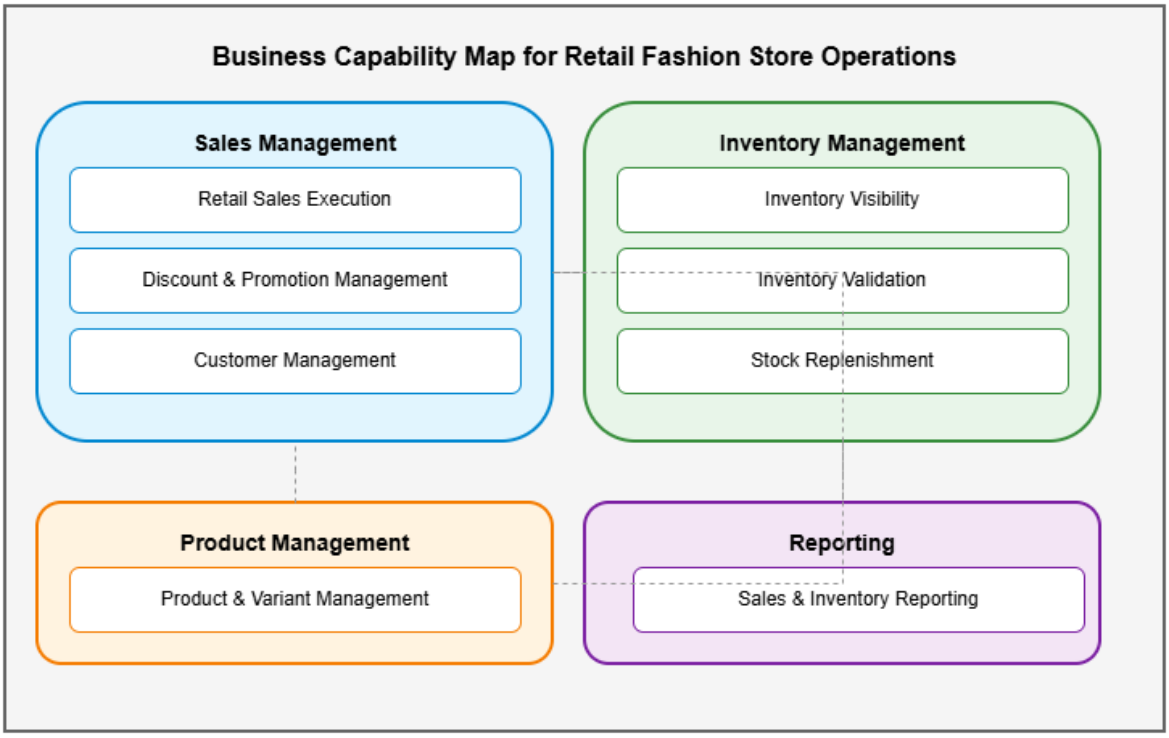

Figure 1: Business Capability Map

1.2.2 Capability-to-Process Alignment

Each business capability is realized through one or more business processes described in later sections:

- Retail Sales Execution and Inventory Validation are implemented through the TO-BE sales process modeled using BPMN.
- Stock Replenishment is supported by purchasing and inventory update processes.
- Product & Variant Management underpins both sales and inventory operations.

This alignment ensures traceability from high-level business intent to operational execution.

1.3 AS-IS Business Problems

In the current state, the retail fashion store operates with largely manual management practices and without an integrated information system to support sales, inventory, or financial operations. Core activities such as recording sales, tracking inventory levels, and summarizing financial performance are handled through handwritten notes, informal records, or basic spreadsheets.

This manual approach results in fragmented and inconsistent operational data. Sales information is not captured in a structured or centralized manner, making it difficult to consolidate transactions over time. Inventory records are similarly unreliable, as stock levels are often estimated or reconciled only during periodic physical counts rather than continuously monitored.

From a financial perspective, the absence of system-supported transaction records creates significant challenges in revenue tracking and financial reconciliation. Daily or monthly sales figures must be reconstructed manually, increasing the risk of errors and delays. Discrepancies between recorded sales and actual cash flow are difficult to identify and resolve, reducing management's confidence in financial data.

Inventory auditing and stocktaking are also inefficient under the AS-IS model. Physical counts require substantial time and effort, disrupt normal operations, and often reveal inconsistencies that cannot be traced back to specific sales or inventory movements due to the lack of historical transaction data.

Business Impact of the AS-IS State:

Problem	Business Impact
Manual sales recording	High risk of errors and data inconsistency
Fragmented inventory tracking	Low confidence in stock figures
Manual financial reconciliation	Delayed and inaccurate financial insights
Inefficient stocktaking	High labor cost and operational disruption
Lack of structured reporting	Poor support for managerial decisions

Table 2: Business Impact of the AS-IS State

Overall, the AS-IS state highlights that manual management practices are insufficient for maintaining operational control and financial transparency in a retail fashion environment. While these practices may function at a very small scale, they do not support reliable reporting, auditing, or informed decision-making as the business grows.

1.4 Scope & Assumptions

1.4.1 Scope Definition

The scope of this project covers only the essential business capabilities required to support daily retail sales, inventory visibility, and basic financial tracking. Advanced or non-critical capabilities are deliberately excluded from the current phase and may be considered in future enhancements.

1.4.2 Key Assumptions

The architectural design is based on the following assumptions:

- The store operates a single physical retail location with one warehouse.
- Sales are conducted directly at the counter, supported by a centralized ERP system.
- Product pricing already includes tax, eliminating the need for advanced tax configuration.
- Promotions and discounts are applied directly during the sales
- Customer data is maintained to support re
- The system prioritizes operational accuracy and usability over financial complexity.

These assumptions ensure that the architecture remains lean, realistic, and aligned with actual retail practices, while still supporting customer engagement and promotional activities.

Capability Area	In Scope	Out of Scope	Notes
Retail Sales Management	✓		Sales order creation at store level
Product & Variant Management	✓		Size-color variants
Inventory Visibility	✓		Variant-level stock view
Inventory Validation	✓		System-supported stock check
Discounts & Promotions	✓		Applied at sales order level
Customer Management (CRM)	✓		Loyalty / repeat customers
Financial Recording	✓		Tax included in product price
Purchasing	✓		Basic PO and receipt handling
Automation Rules		✓	Future phase
Automated Invoicing & Payment		✓	Future phase
Multi-warehouse Operations		✓	Single warehouse only
Advanced Accounting & Tax		✓	Including in products prices
E-commerce Integration		✓	Physical store only

Table 3: Scope Definition (In-scope vs Out-of-scope)

1.5 TO-BE Business Process

1.5.1 Purpose of the TO-BE Process

The TO-BE business process aims to transform the current manual retail operation into a system-supported and disciplined sales workflow, while remaining suitable for a small fashion store environment. The focus is on improving sales accuracy, data consistency, and usability at the point of sale without introducing unnecessary operational complexity.

This process represents the core value-creating activity of the enterprise and serves as the foundation for the subsequent Application, Data, and Technology Architecture layers.

1.5.2 TO-BE Business Process

In the TO-BE state, retail sales are executed through a centralized ERP system that supports sales staff during customer transactions. The process ensures that product selection, customer management, promotions, and stock validation are handled consistently and recorded in a structured manner. At a high level, the process consists of the following steps:

1. A customer selects products at the store.
2. Sales staff creates a sales order in the system.
3. Customer information is selected or registered.
4. Discounts or promotions are applied if applicable.
5. The system validates stock availability for the selected product variants.
6. The sales order is confirmed and recorded.

The process is designed to be lean, minimizing manual checks and enforcing business rules through the system.

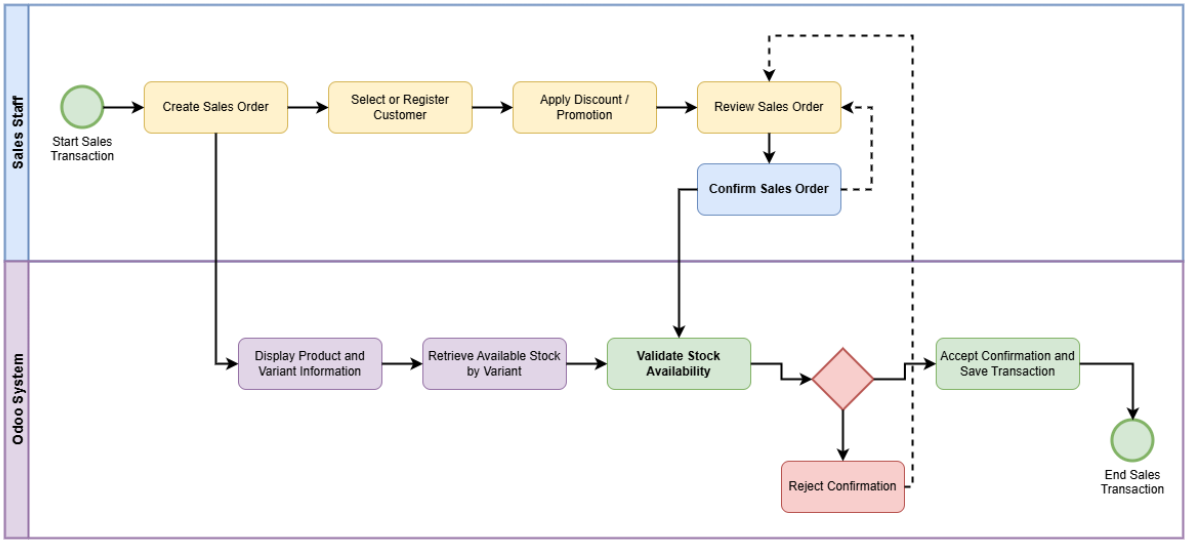

Figure 2: BPMN Diagram Specification (TO-BE)

2 Application Architecture

2.1 Application Landscape

This section describes the application landscape that supports the TO-BE retail sales process defined in the Business Architecture layer. The objective is to identify the core application components, both standard and customized, and to explain how they collectively support lean retail operations in a physical fashion store.

The architecture adopts Odoo 18 as a single, integrated ERP platform, avoiding fragmented systems or standalone applications. Customization is applied selectively through extension modules to address two key needs:

1. enforcing business rules at the point of sale, and
2. simplifying the user interface for frontline sales staff.

This application-driven approach ensures operational consistency, ease of use, and long-term maintainability.

2.1.1 Core Application Components

2.1.1.a Standard Odoo Modules (Core Applications)

The following standard Odoo modules are used and configured to support in-scope business capabilities:

Module	Primary Responsibility
Sales	Creation and management of sales orders at the retail counter
Inventory	Management of product stock and variant-level availability
Purchase	Handling of basic purchasing and goods receipt
Contacts (CRM)	Storage and management of customer and loyalty information
Accounting (Basic)	Recording of sales transactions (tax included in price)

Table 4: Standard Odoo Modules Used in the System

These modules provide the foundational ERP functionality and are used without modification to core source code.

2.1.1.b Custom Application Modules

To adapt the ERP system to the operational reality of a small retail fashion store, two custom application modules are introduced:

Custom Module	Architectural Role
`sale_quick_checkout`	Extends the Sales module with stock availability validation and confirmation-time business rules
`lean_sales_ui` (Lean UI Module)	Simplifies the sales interface by hiding non-essential fields and options for sales staff

Table 5: Custom Application Modules

The Lean UI module focuses exclusively on usability and operational efficiency, ensuring that sales staff interact only with fields and actions required for daily retail transactions. It does not introduce new business logic, but instead complements the functional customization provided by `sale_quick_checkout`.

2.1.2 Logical Grouping of Application Functions

From an architectural perspective, the application components can be organized into the following logical groups:

1. **Sales Execution Layer**

   - **Sales**
   - `sale_quick_checkout`
   - `lean_sales_ui`

   Supports sales order creation, customer selection, discount application, stock validation, and order confirmation at the retail counter.

2. **Inventory Support Layer**

   - **Inventory**

   Provides stock visibility and supports validation of product variant availability.

3. **Supporting Business Services Layer**

   - **Purchase**
   - **Contacts**
   - **Accounting (Basic)**

   Supports inventory replenishment, customer management, and basic financial recording.

This grouping clarifies the separation of concerns while preserving tight integration between components.

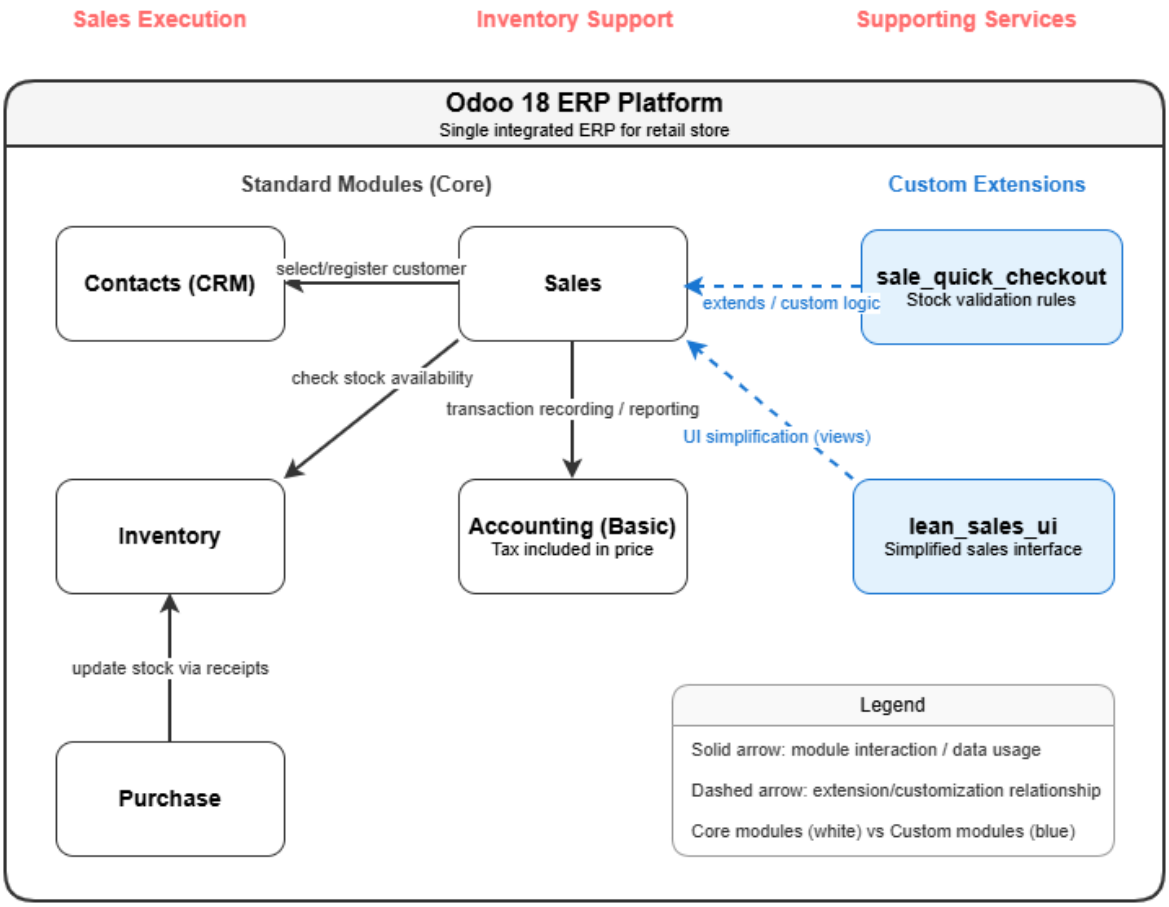

Figure 3: Application Landscape Diagram

The application architecture is centered on Odoo 18 as a single integrated ERP platform. Core modules support sales execution, inventory management, purchasing, customer management, and basic accounting. Custom extension modules (`sale_quick_checkout` and `lean_sales_ui`) extend the Sales module to enforce stock validation rules and simplify the user interface for retail counter operations.

2.2 Customization Strategy & Application Extensions

To meet the needs of a retail fashion store while keeping the system maintainable, the solution follows a configuration-first, extension-second strategy. Standard Odoo modules are reused wherever possible, and custom behavior is introduced only when it directly supports in-scope business capabilities defined in the Business Architecture layer. Customization is implemented through separate add-on modules rather than modifying Odoo core code. This approach ensures:

- clear separation between standard ERP functionality and project-specific logic
- easier upgrades and maintenance,
- controllable scope and reduced risk of regression.

2.2.1 Custom Modules and Their Architectural Roles

Two custom extension modules are introduced to support the TO-BE retail sales process:

2.2.1.a `lean_sales_ui`

Architectural intent: Improve usability for sales staff by presenting only necessary fields and actions during retail sales.

What it changes (type of customization):

- View inheritance to hide non-essential fields, tabs, and options in the Sales interface.
- Menu and action simplification aligned with the retail counter workflow.

What it does NOT do:

- Does not introduce new business logic.
- Does not modify transactional workflows.
- Only affects presentation and user interaction.

Business value:

- Faster sales order creation at the retail counter.
- Reduced training time for new sales staff.
- Lower risk of incorrect data entry caused by interface complexity.

2.2.1.b `sale_quick_checkout` – Sales Extensions for Stock Validation

Architectural intent: Enforce stock visibility and validation at the point of sale to ensure that retail sales transactions are consistent and reliable.

Key application extensions (high level):

- Adds variant-level stock visibility to sales order lines.
- Validates stock availability during user interaction (onchange) and at transaction confirmation time.

Customization mechanisms:

- Model inheritance on Sales-related objects to inject computed stock information and validation rules.
- Business rule enforcement at order confirmation to prevent inconsistent sales transactions.

Business value:

- Enables sales staff to make decisions based on system-provided stock availability.
- Prevents invalid sales confirmations when stock conditions are not satisfied.
- Improves the reliability of operational data for reporting and reconciliation.

2.2.2 Mapping Customizations to Business Capabilities

The custom modules directly support the in-scope capabilities established earlier:

Business Capability	Odoo Core Support	Custom Extension
Retail sales execution	Sales	`lean_sales_ui` (UI simplification)
Customer selection / loyalty	Contacts + Sales	`lean_sales_ui` (guided UX)
Discount / promotion application	Sales	Configuration-based (optional UI hiding / exposure)
Inventory visibility at sale	Inventory	`sale_quick_checkout` (availability surfacing)
Stock validation at confirmation	Sales + Inventory	`sale_quick_checkout` (business rule enforcement)

Table 6: Mapping Customizations to Business Capabilities

This mapping ensures traceability from business needs to application design decisions.

The customization strategy ensures that the solution remains lean and retail-focused while preserving the strengths of a standard ERP platform. Separating UI simplification (`lean_sales_ui`) from business rule enforcement (`sale_quick_checkout`) provides a clean architectural structure:

- UI changes improve usability without affecting transactional integrity.
- Business logic extensions enforce discipline without overcomplicating the workflow.
- Standard Odoo modules remain the foundation, ensuring maintainability and scalability.

3 Data Architecture

This section identifies the core business data entities required to support the TO-BE retail sales process. The data entities are described at a conceptual level and grouped into Master Data and Transaction/Operational Data to improve clarity and traceability.

3.1 Core Data Entities

3.1.1 Master Data

Master data represents relatively stable business objects reused across multiple business processes.

- **Product** Defines a fashion item at a general level for cataloging and reporting.
- **Product Variant** Represents a specific size–color combination and acts as the primary unit of sale and inventory control.
- **Customer** Represents individuals who purchase products from the store. Customer data supports retail transactions, loyalty tracking, and customer-based reporting.
- **Vendor** Represents suppliers who provide products to the store. Vendor data supports procurement and inventory replenishment processes.

Note: At the implementation level, both Customer and Vendor are managed using the Partner model with different business roles.

3.1.2 Transaction Data

Transaction and operational data represent business events and measurable states that change frequently.

- **Sale Order** Captures a retail sales transaction initiated at the counter.
- **Sale Order Line** Represents individual product variants and quantities within a sale order.
- **Purchase Order** Records procurement transactions with vendors and serves as the source of incoming inventory.
- **Inventory Stock (Stock Quant)** Represents the current on-hand quantity of each product variant in the warehouse.

Summary Table

Data Group	Entity	Business Meaning	Role in the Architecture
Master Data	Product	General product definition	Catalog grouping
Master Data	Product Variant	Size–color combination	Unit of sale and inventory
Master Data	Customer	Buyer identity	Sales and loyalty management
Master Data	Vendor	Supplier identity	Procurement
Transaction Data	Sale Order	Retail sales transaction	Sales record
Transaction Data	Sale Order Line	Items in a sale	Stock validation point
Transaction Data	Purchase Order	Procurement transaction	Source of inventory
Operational Data	Stock Quant	On-hand quantity	Inventory control

Table 7: Core Data Entities by Data Type

3.2 Data Relationships Traceability

3.2.1 Conceptual Relationship Model

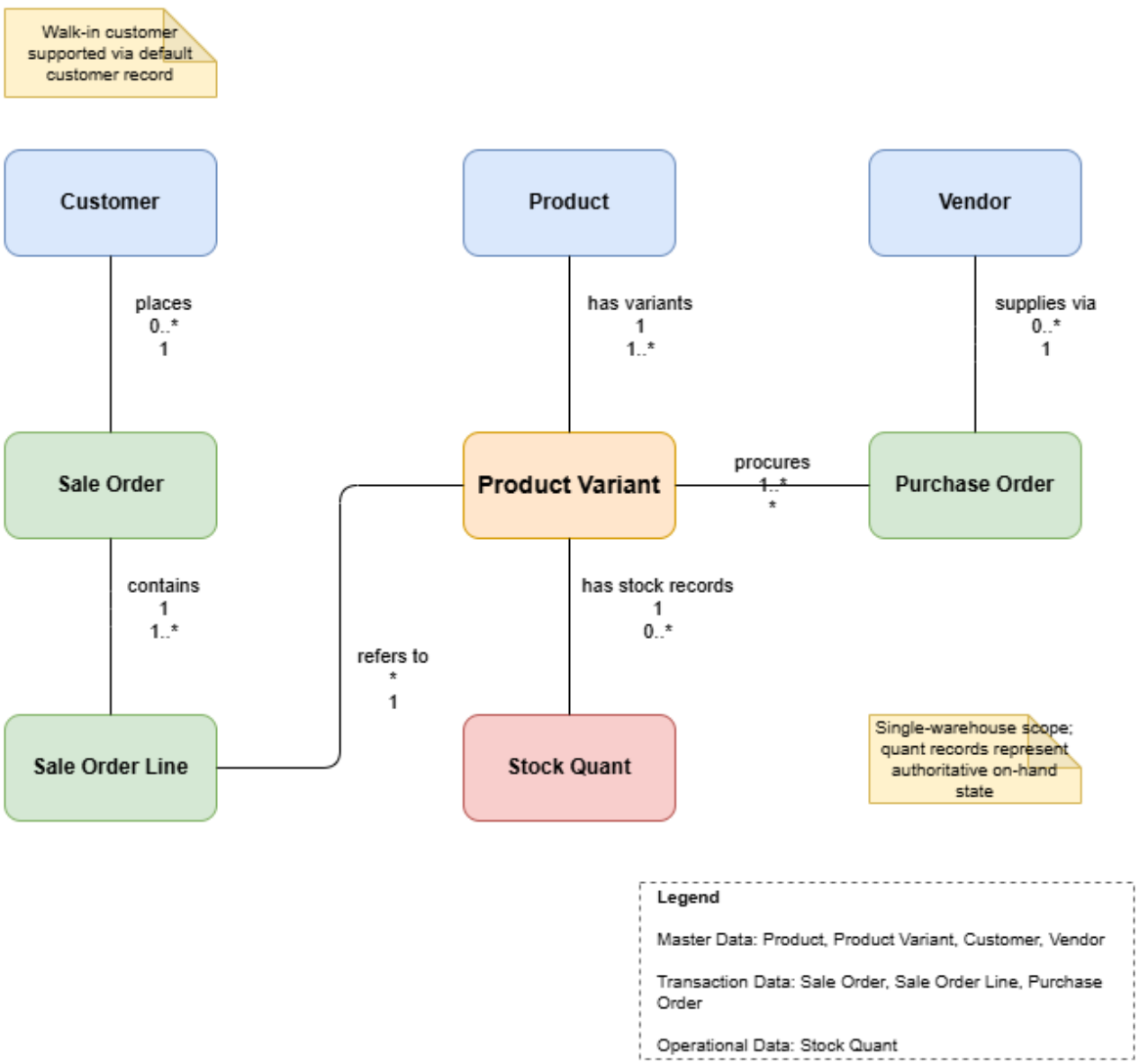

Figure 4: Conceptual Data Model

The model connects retail sales (Customer → Sale Order → Sale Order Line → Product Variant), procurement (Vendor → Purchase Order → Product Variant), and inventory state (Product Variant → Stock Quant). Product Variant is the operational unit for both sales and inventory control in the single-warehouse retail scope.

3.2.2 Traceability to Business Capabilities

This conceptual data model ensures end-to-end traceability across the retail value chain.

- **Retail Sales Traceability**
  Customer → Sale Order → Sale Order Line → Product Variant
  This enables accurate sales reporting by customer, product, and variant.

- **Inventory Visibility**
  Product Variant → Stock Quant
  This enables real-time stock visibility and supports controlled sales decisions at the retail counter.

- **Procurement Traceability**
  Vendor → Purchase Order → Product Variant
  This supports replenishment history tracking and supplier-based analysis.

3.3 Data Governance & Data Quality Principles

This section defines the data governance principles applied to ensure data consistency, accuracy, and reliability across sales, inventory, and procurement operations. The principles focus on operational data quality rather than complex governance frameworks, aligning with the lean scope of a single-store retail environment.

3.3.1 Data Governance Principles

The system follows a set of lightweight governance rules to maintain control over core business data while avoiding unnecessary administrative overhead.

3.3.1.a Principle 1: Single Source of Truth

All core business data is maintained within the ERP platform, ensuring that:

- product, variant, customer, vendor, and inventory data are not duplicated across systems,
- sales, purchasing, and inventory reporting rely on the same authoritative data sources.

This principle prevents inconsistencies caused by manual spreadsheets or parallel record keeping.

3.3.1.b Principle 2: Role-Based Data Responsibility

Data access and modification rights are aligned with operational responsibilities:

- Sales staff manage sales transactions.
- Inventory and purchasing roles manage stock levels and purchase orders.
- Administrative users maintain master data such as products, variants, customers, and vendors.

This separation reduces the risk of unauthorized or accidental data changes.

3.3.1.c Principle 3: System-Enforced Business Rules

Critical business rules are enforced by the application rather than manual checks. For example, product variant availability is validated during sales transactions, and confirmation is blocked when required constraints are not satisfied. This ensures consistent rule enforcement across all operations.

3.3.1.d Principle 4: Controlled Data Lifecycle

Core data entities follow defined lifecycles:

- Master data (products, variants, customers, vendors) is created infrequently and reused across processes.
- Transaction data (sale orders, purchase orders) becomes immutable once confirmed.
- Operational data (stock quantities) reflects the current system state and is updated only through authorized processes.

3.3.1.e Principle 5: Scope-Appropriate Governance

The governance model is designed to support daily retail operations without introducing unnecessary complexity. Advanced auditing, regulatory compliance, and historical versioning are considered out of scope for the current implementation.

3.3.2 Data Quality Dimensions

To ensure usable and reliable data, the following data quality dimensions are applied:

Dimension	Description	Application in the System
Accuracy	Data correctly represents real-world events	Stock quantities reflect actual on-hand inventory
Consistency	Same data used across processes	Sales and inventory share the same product variant records
Completeness	Required data is always present	Sale orders require customer and product variant information
Timeliness	Data is up to date	Inventory availability is checked in real time
Traceability	Data can be traced across processes	Sales and purchase records are linked to product variants

Table 8: Data Quality Dimensions and Applications

4 Technology Architecture

4.1 Technology Architecture Overview

This section describes the technology environment that supports the application and data architectures defined in previous layers. The Technology Architecture focuses on how the system is deployed and operated in a physical retail store, emphasizing reliability, simplicity, and ease of maintenance.

The design intentionally avoids complex distributed infrastructure in order to align with the operational reality of a single-store retail environment.

The system is deployed as a locally hosted ERP solution, with all application and data components running within a controlled on-premise environment. This approach ensures stable performance, reduced dependency on external services, and straightforward system administration.

The technology stack includes:

- a centralized application server hosting the ERP platform,
- a relational database for transactional and master data,
- client devices accessing the system through a web interface over a local network.

4.1.1 Core Technology Components

4.1.1.a Application Server

The ERP application runs on a dedicated application server hosting **Odoo 18**. The server is responsible for:

- executing business logic and custom application extensions,
- rendering the user interface for sales and administrative users,
- enforcing validation rules and access control policies.

Custom modules are deployed alongside standard Odoo modules within the same runtime environment.

4.1.1.b Database Server

A relational database management system is used to store all business data, including:

- master data (products, variants, customers, vendors),
- transaction data (sales orders and purchase orders),
- operational data (inventory stock quantities).

The database acts as the single source of truth and is tightly integrated with the ERP application.

4.1.1.c Client Layer

End users interact with the system through standard web browsers on:

- desktop or laptop computers at the retail sales counter,
- administrative workstations used for product, inventory, and purchasing management.

No dedicated client installation is required, reducing deployment complexity and maintenance effort.

4.1.2 Network and Access Model

The system operates within a local area network (LAN) environment:

- the application server and database server communicate internally,
- client devices access the application via HTTP over the local network.

This network model provides sufficient performance and security for a small retail store while remaining easy to manage.

4.1.3 Technology Architecture Diagram

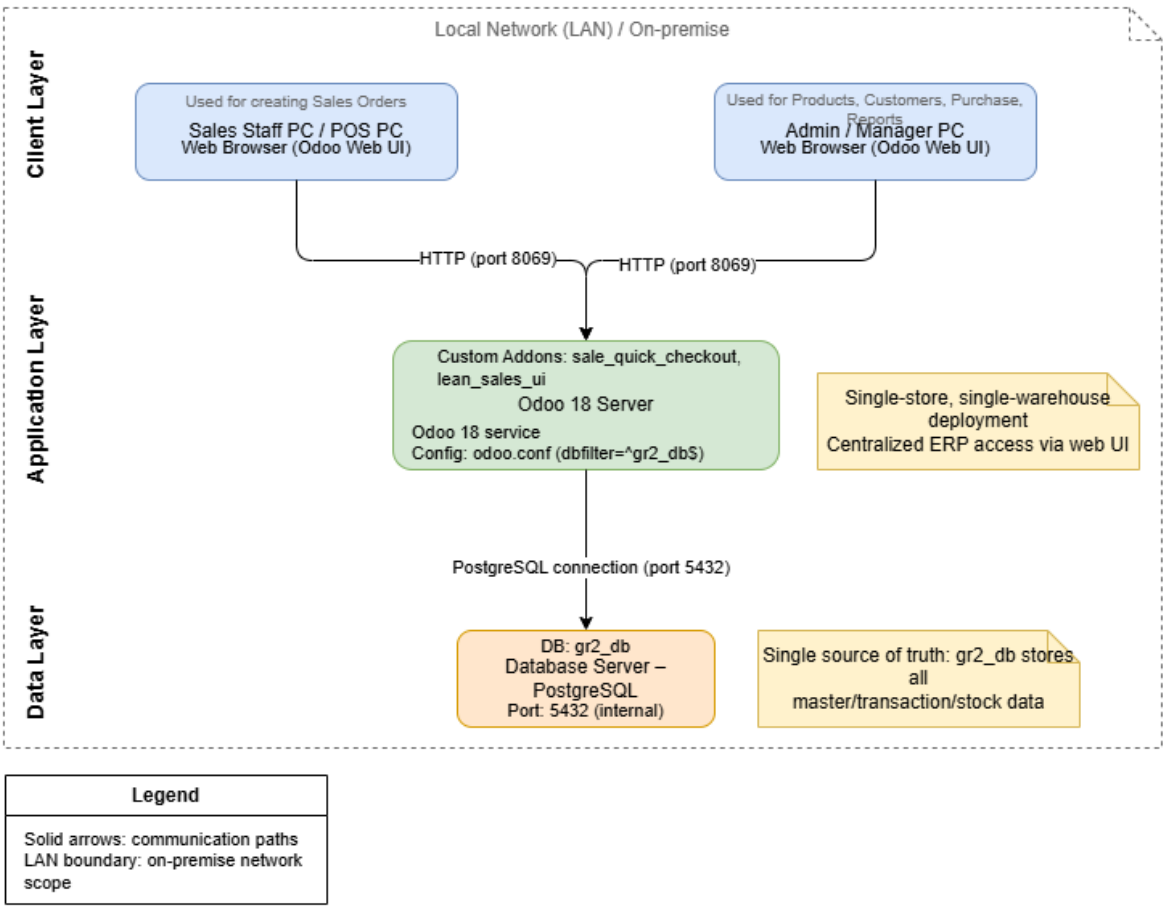

Figure 5: Technology Architecture Overview

This figure illustrates the high-level technology architecture of the system.
The diagram includes:

- client devices (Sales PC and Admin PC),
- the application server running Odoo 18 with custom modules,
- the database server hosting the relational database,
- the local network connecting all components.

The purpose of this diagram is to show how users, applications, and data are physically arranged and how they interact at a high level. Detailed operating system or hardware specifications are intentionally excluded.

4.1.4 Architectural Characteristics

The technology architecture exhibits the following characteristics:

- **Simplicity:** Minimal number of components to reduce operational overhead.
- **Reliability:** Local deployment ensures stable performance during store operations.
- **Maintainability:** Centralized application and database simplify updates and backups.
- **Scalability (within scope):** The architecture supports additional users or minor feature expansion without redesign.

4.2 Deployment and Configuration

4.2.1 Deployment Model

The system follows a centralized on-premise deployment model, where all core components are hosted within the same local environment. In this model:

- The ERP application and custom modules run on a single application instance.
- The database is hosted on a local database server.
- Users access the system through standard web browsers over a local network.

This approach minimizes infrastructure complexity and is well suited for a physical retail store with a limited number of users.

4.2.2 Application Deployment

4.2.2.a Odoo Platform Deployment

The ERP platform is deployed using Odoo 18. Standard Odoo modules and project-specific custom modules are loaded through explicitly defined add-on paths. Custom modules are isolated from standard modules to preserve maintainability and upgrade compatibility.

The application is exposed through a single HTTP endpoint that is accessible from authorized client devices within the store network.

4.2.2.b Custom Module Deployment

Custom functionalities are implemented as independent add-on modules, including:

- `sale_quick_checkout`
- `lean_sales_ui`

These modules are installed and managed using Odoo's standard module management mechanism, allowing custom behavior to be enabled or modified without altering the core ERP codebase.

4.2.3 Database Deployment

A relational database management system is used to store all business data, including master data, transactional data, and operational inventory state. A single dedicated database is used for the store to ensure data isolation and simplify backup and recovery procedures.

Direct user access to the database is restricted. All data interactions occur through the ERP application layer to preserve data integrity and security.

4.2.4 Configuration Strategy

4.2.4.a Application Configuration

Key application configuration principles include:

- restricting the system to a single project database,
- explicitly defining add-on paths for both standard and custom modules,
- centralizing configuration parameters in a single configuration file.

This strategy reduces configuration errors and improves operational transparency.

4.2.4.b Access Control Configuration

Access to the system is governed through role-based user accounts. Sales staff are granted access only to interfaces required for retail operations, while administrative users manage master data, purchasing, and system configuration.

4.2.5 Deployment Diagram

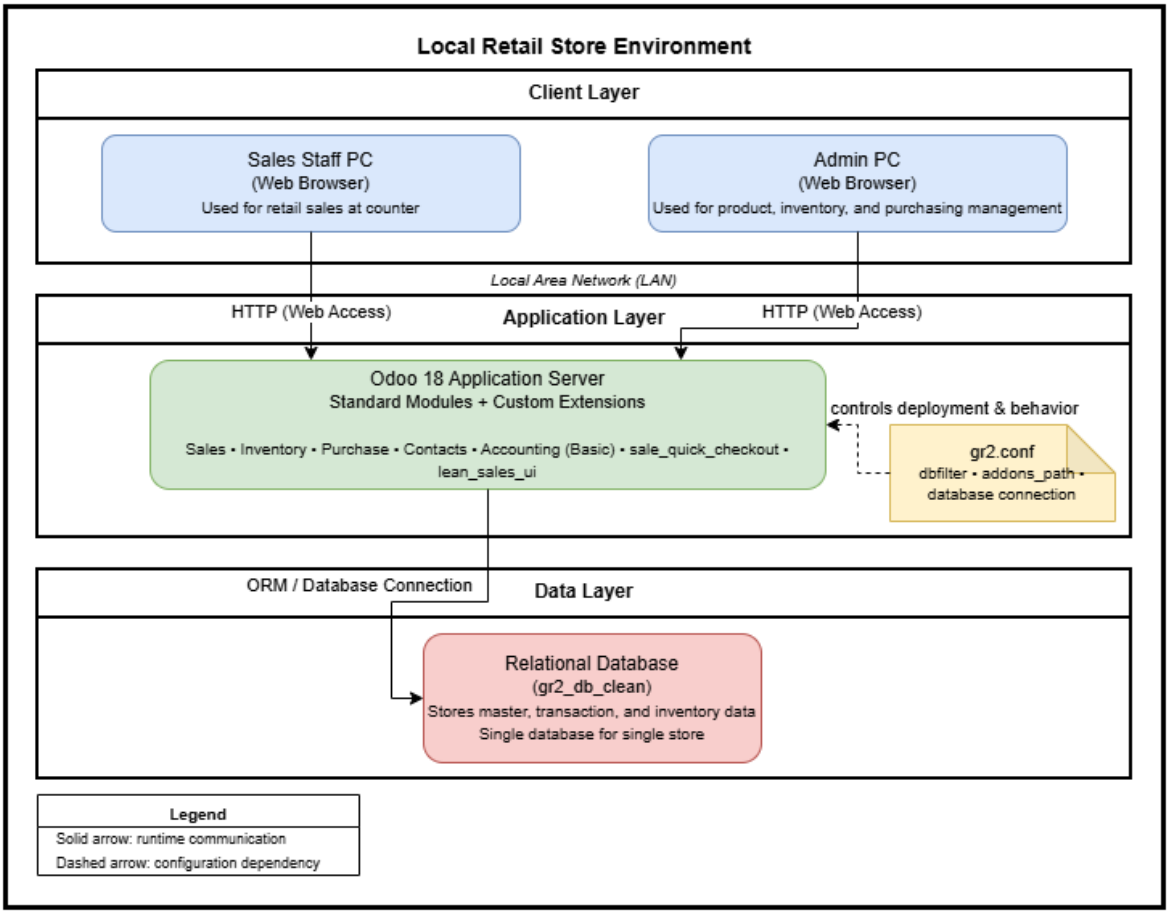

Figure 6: Deployment and Configuration Diagram

This figure illustrates the deployment and configuration overview, showing client devices, the Odoo application instance, the database instance, and the configuration file that controls system behavior.

5 Architecture Summary and Conclusion

5.1 Architecture Summary

This project presents a *lean enterprise architecture* for a single-store retail fashion business, designed to address the limitations of manual operations while avoiding unnecessary system complexity. The architecture follows a clear four-layer Enterprise Architecture (EA) structure, ensuring traceability from business objectives to technology implementation.

At the **Business Architecture** layer, the core focus is the retail sales process at the physical counter. The TO-BE process emphasizes structured sales execution, improved customer handling, controlled discount application, and system-enforced stock validation. The process is intentionally kept simple to reflect real-world retail operations and to reduce reliance on manual checks and individual experience.

The **Application Architecture** layer centers on Odoo 18 as a single integrated ERP platform. Standard ERP modules are reused to provide core functionality, while customization is applied selectively through extension modules. The separation between functional extensions (`sale_quick_checkout`) and interface simplification (`lean_sales_ui`) ensures that business rules are enforced consistently while maintaining a user-friendly experience for sales staff.

The **Data Architecture** layer defines a concise and variant-centric data model. Product Variant serves as the operational unit connecting sales, purchasing, and inventory control. Clear relationships between customers, vendors, transactions, and stock records enable end-to-end traceability without introducing unnecessary technical detail. Lightweight data governance principles ensure data quality, consistency, and operational reliability within the defined scope.

Finally, the **Technology Architecture** layer provides a practical deployment model suitable for a small retail store. A centralized, locally hosted system minimizes infrastructure complexity while delivering stable performance. Configuration-driven deployment allows system behavior to be controlled without modifying core application code, supporting maintainability and future evolution.

5.2 Key Architectural Decisions

Several key architectural decisions underpin the overall solution:

- Adoption of a single ERP platform to avoid data fragmentation.
- Preference for configuration and extension over core modification.
- Variant-level inventory control to support accurate retail sales decisions.
- Lightweight governance and deployment aligned with store-scale operations.

These decisions collectively balance functional control, usability, and maintainability.

5.3 Alignment with Business Objectives

The architecture directly supports the original business objectives of the project:

- Improving sales accuracy and consistency at the retail counter.

- Reducing manual effort in stock checking and transaction recording.
- Enabling basic reporting and inventory visibility for management.
- Maintaining ease of deployment and day-to-day operation.

By aligning architectural design with actual store workflows, the system delivers practical value rather than theoretical complexity.

5.4 Limitations and Future Evolution

The current architecture is intentionally scoped to a single-store environment and does not include advanced automation such as automatic delivery validation, invoicing, or payment workflows. These capabilities may be introduced in future phases without redesigning the architecture, leveraging the modular nature of the ERP platform.

Potential future enhancements include:

- Extended inventory automation.
- Deeper financial processing.
- Multi-store or multi-warehouse support.
- Integration with external sales channels.

5.5 Conclusion

In conclusion, this project demonstrates that a well-structured, lean enterprise architecture can effectively support retail operations when aligned with real business needs. By combining disciplined business process design, selective application customization, clear data modeling, and practical deployment decisions, the architecture provides a solid and extensible foundation for retail management.

The resulting system achieves a balance between control and simplicity, making it suitable for immediate use while remaining adaptable to future growth.

Appendix

A Odoo Configuration File

This appendix documents the main configuration parameters used to deploy and operate the ERP system. The configuration file centralizes deployment behavior, database access, and module loading paths.

A.1 Configuration Overview

The system is configured using a single configuration file (`gr2.conf`), ensuring predictable application behavior and simplified maintenance.

The configuration file is responsible for:

- Restricting database access to the project database only
- Defining add-on paths for standard and custom modules
- Controlling application startup and runtime behavior

A.2 Configuration File Content (Excerpt)

```
[options]
; Database settings
db_host = localhost
db_port = 5432
db_user = lenovo
db_password = ****
db_name = gr2_db_clean

; Addons path (standard addons + project addons)
addons_path = D:\odoo-18.0\odoo-18.0\addons,
              D:\odoo-18.0\odoo-18.0\addons\codenew

; Server settings
http_port = 8069

; Database filter
dbfilter = ^gr2_db$

; Admin password for database operations
admin_passwd = ****

; Logging
log_level = info
logfile = D:\odoo-18.0\odoo-18.0\gr2.log

; PDF report support
wkhtmltopdf_path = C:\Program Files\wkhtmltopdf\bin\wkhtmltopdf.exe
```

A.3 Configuration Notes

- A single database is used to isolate project data.
- Custom modules are loaded through a dedicated add-ons directory.
- Database filtering prevents accidental access to unintended databases.
- Logging is enabled to support debugging and operational monitoring.

B Application Module List

This appendix lists the application modules used in the system, categorized by standard ERP functionality and project-specific extensions.

B.1 Standard Odoo Modules

Module	Purpose
Sales	Retail sales order creation and management
Inventory	Product stock and variant-level availability
Purchase	Basic procurement and goods receipt
Contacts	Customer and vendor information management
Accounting (Basic)	Transaction recording and basic reporting

Table 9: Standard Odoo Modules

These modules provide core ERP functionality and are used without modification to their source code.

B.2 Custom Modules

Module	Description	Architectural Role
`sale_quick_checkout`	Extends sales process with stock availability validation	Business rule enforcement
`lean_sales_ui`	Simplifies sales interface by hiding non-essential fields	User experience optimization

Table 10: Custom Application Modules

Custom modules are implemented as independent add-ons to ensure maintainability and upgrade compatibility.

C Screenshots and Evidence Notes

This appendix describes the screenshots included as implementation evidence. Screenshots demonstrate the alignment between the enterprise architecture design and the implemented system.

- **Sales Order Interface (Lean UI)**:

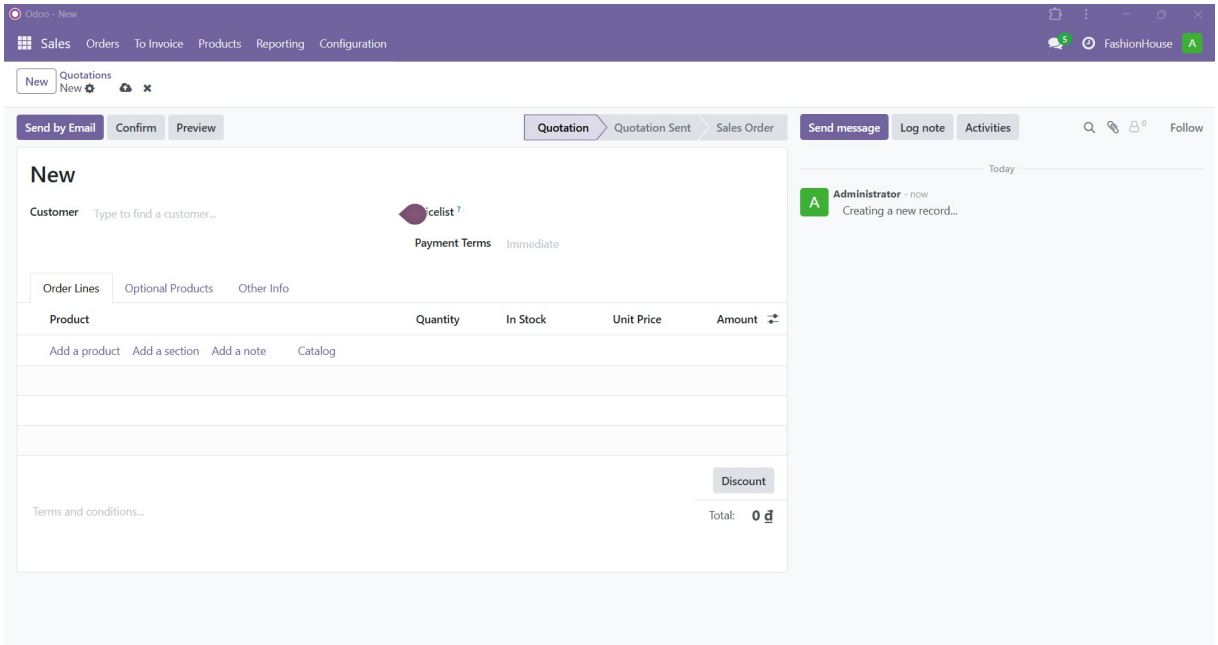

Figure 7: Sales Order Interface (Lean UI) Screen

- **Stock Availability Display**:

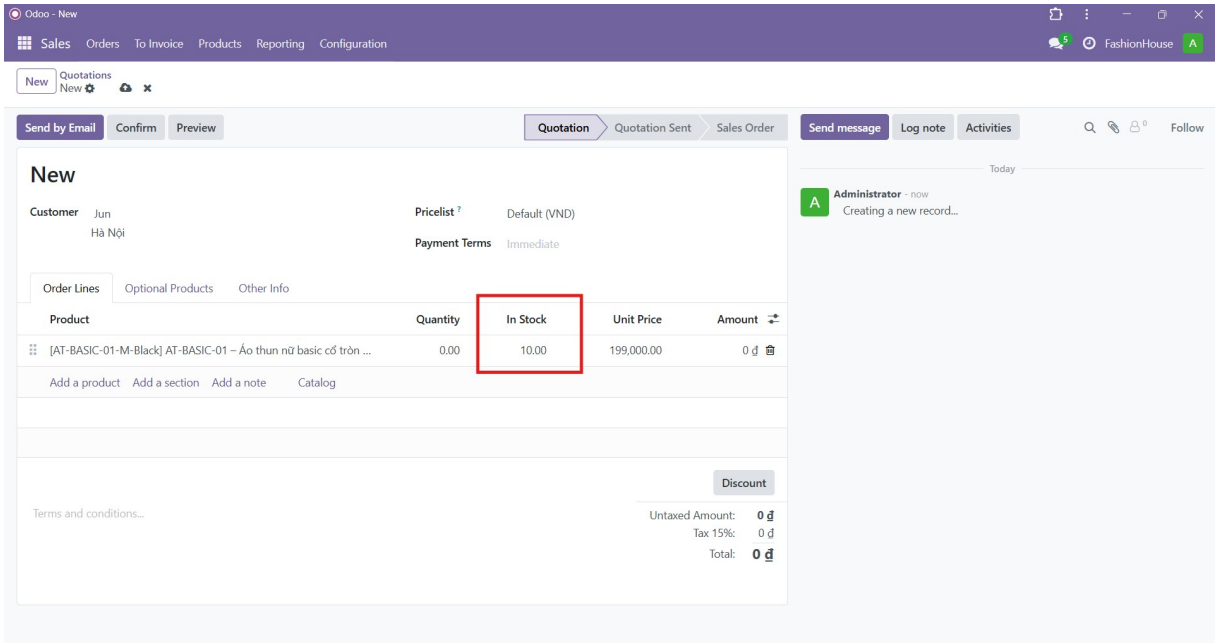

Figure 8: Stock Availability Display Screen

- **Stock Validation Warning**:

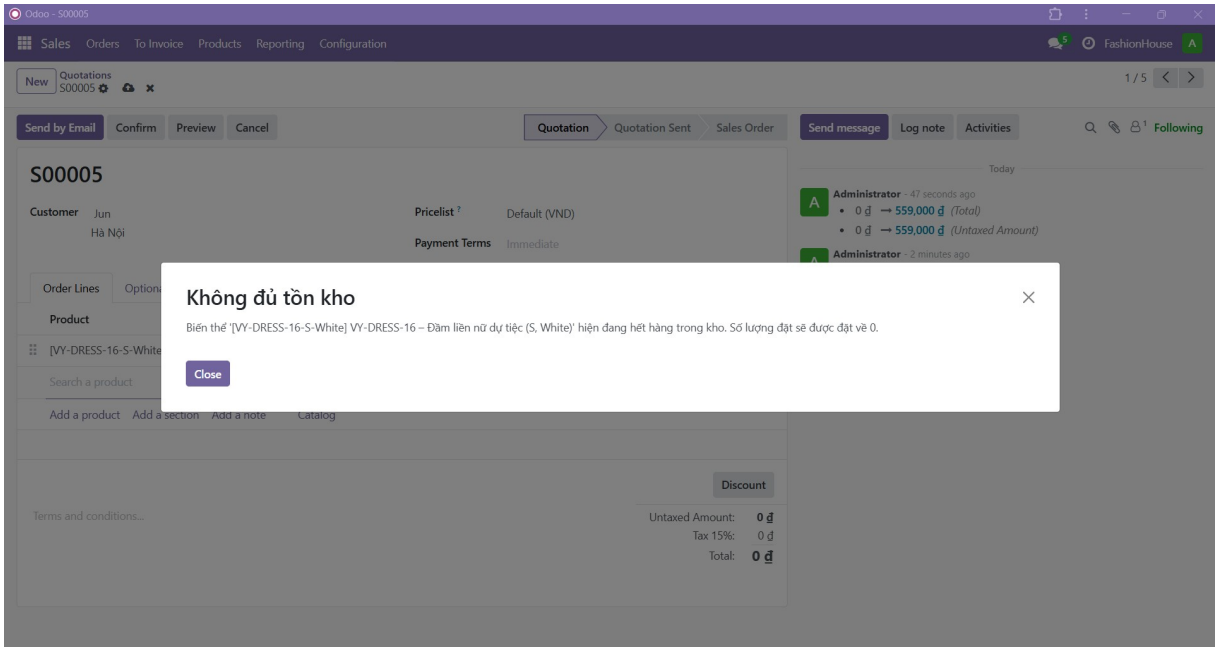

Figure 9: Stock Validation Warning Screen

- **Installed Module List**:

Module Name	Author	Website	Latest Ver...	Status
Sales	Odoo S.A.	https://www.odoo.com/app/sales	18.0.1.0	Installed
Purchase	Odoo S.A.	https://www.odoo.com/app/purc...	18.0.1.2	Installed
Invoicing	Odoo S.A.	https://www.odoo.com/app/invoi...	18.0.1.3	Installed
Inventory	Odoo S.A.	https://www.odoo.com/app/inven...	18.0.1.1	Installed
GR2 Lean Sales UI	GR2 Project	https://example.com	18.0.1.0.0	Installed
Sale Quick Checkout	GR2 Project	https://example.com	18.0.1.0.0	Installed
Contacts	Odoo S.A.		18.0.1.0	Installed

Figure 10: Installed Module List Screen

- **Database Selection Screen**:

Figure 11: Database Selection Screen

D Scope Clarification

For evaluation purposes, the system scope is clarified as follows:

- Single retail store
- Single warehouse
- Local (on-premise) deployment
- No advanced accounting, invoicing, or payment automation

This clarification ensures that assessment focuses on architectural quality rather than out-of-scope functionality.